

Workforce Management System (WMS)

Low-Level Design Document

This document explains how WMS is built on the inside — the database tables, how the code is organised, what each part does, and how data flows through the system. It is meant for developers who are building or reviewing this project.

1. Database Tables (What Data We Store)

The system has 11 tables. Here is what each one holds:

Table	What it stores
Employee	Personal info (name, email, phone), department, role, join date, and status (Active/Inactive)
UserLogin	Username, hashed password, and role — used to log in
Role	The three user types: Admin, Manager, Employee
Department	Company departments like IT, HR, Finance
Attendance	Daily check-in and check-out times for each employee
Leave	Leave requests — who applied, type, dates, and whether approved
Project	Project name, client, manager, start/end date, and status
Client	External companies that projects belong to
EmployeeProject	Tracks which employees are assigned to which projects
Announcement	Company-wide messages posted by Admin
AuditLog	A record of every important action in the system (who did what and when)

Key things to know:

- Employee is the central table — almost everything links back to it.
- When an employee is deleted, they are usually just set to Inactive so history is kept.
- Passwords are never stored as plain text — they are scrambled using BCrypt.
- Every leave starts as Pending and can only be acted on while it is still Pending.
- EmployeeProject links employees and projects — it records when they were assigned and released.

- AuditLog is written automatically whenever something important happens.

2. How the Code is Organised

The code is split into four projects, each with one job:

Project	What it contains
WMS.Domain	The table/entity classes and repository interfaces — no framework code here
WMS.Application	All the business logic — what actually happens when a user does something
WMS.Infrastructure	Database access code, EF Core setup, JWT token generation
WMS.API	The API endpoints that the Angular frontend calls

The business logic in WMS.Application never directly touches the database — it only talks to interfaces. The actual database code sits in WMS.Infrastructure and can be swapped out without changing the business logic.

3. How Login Works

When a user logs in, their password is checked against the stored hash. If correct, they get a token they use for all future requests:

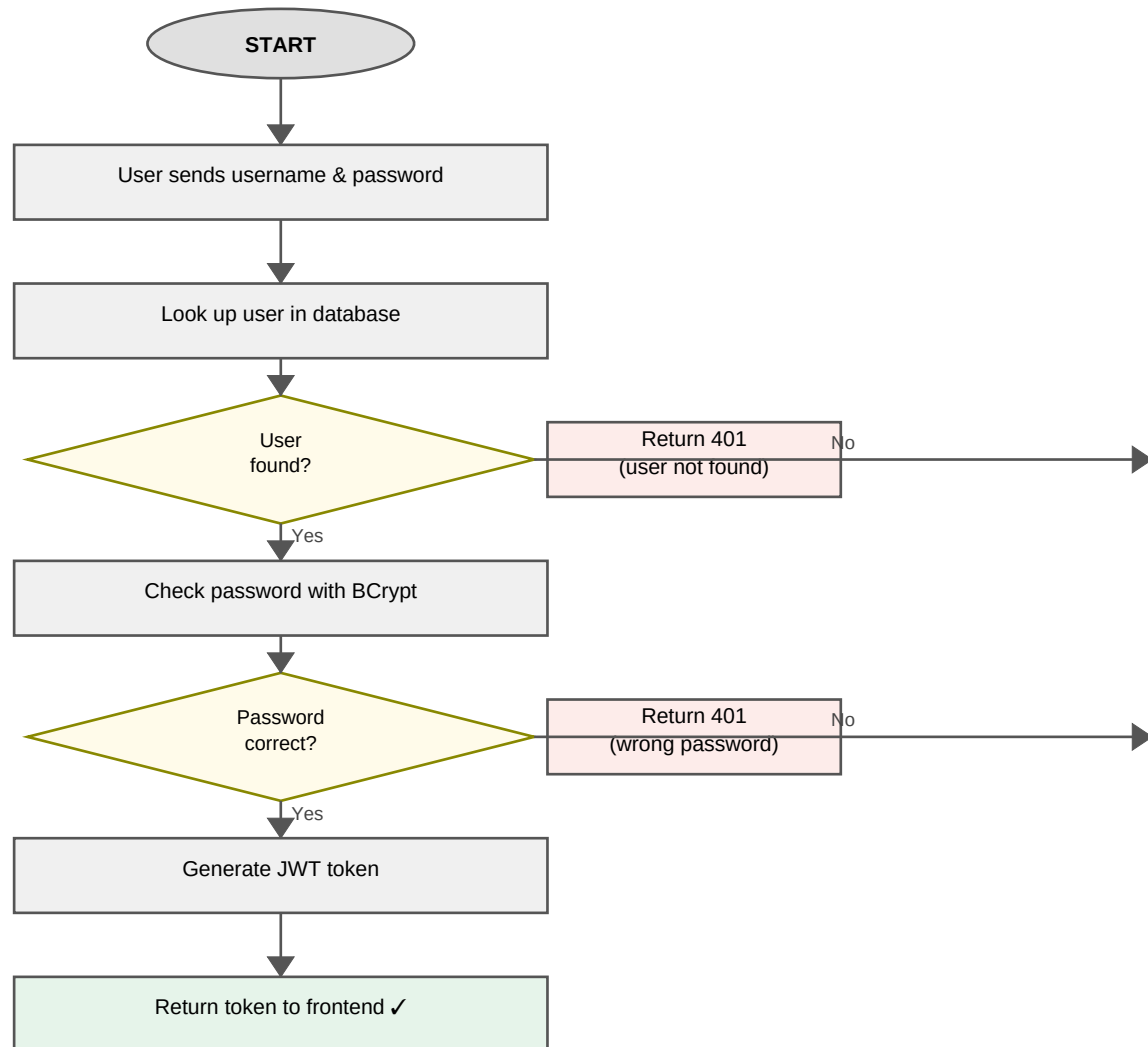


Figure 1 — Login flow

The token contains the user's ID, username, role, and employee ID. The frontend stores it and sends it with every request.

4. How Adding a New Employee Works

Creating an employee is not just one database insert — several things happen automatically:

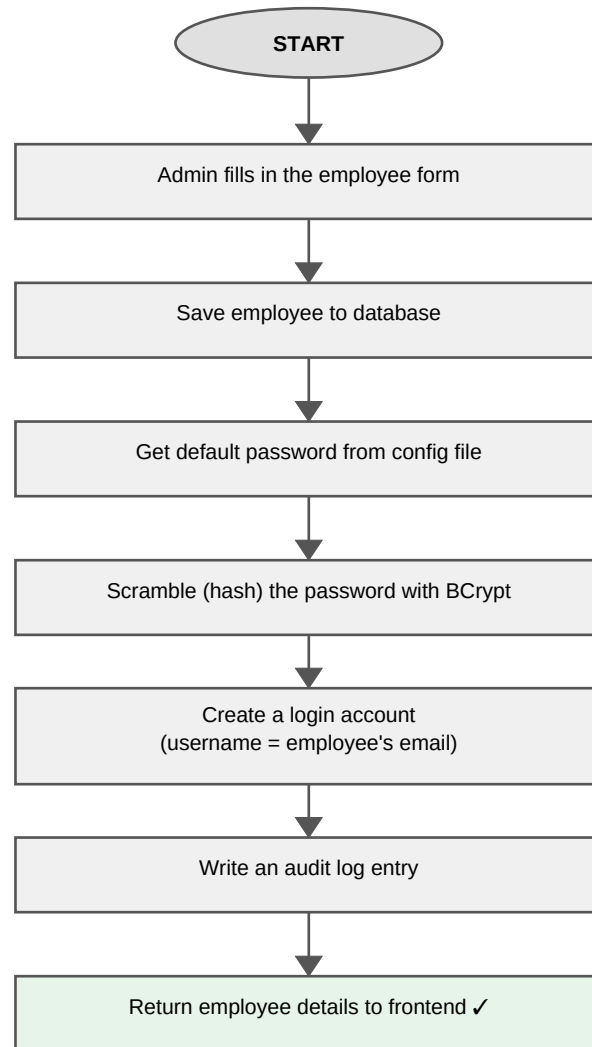


Figure 2 — Employee creation flow

The employee's login username is set to their email. Their starting password comes from the config file, is hashed, and they are prompted to change it on first login.

5. How Check-In and Check-Out Work

Employees check in and out once per day. The system blocks duplicate check-ins and calculates total hours automatically on checkout:



Figure 3 — Check-in (right) and check-out (left) flows

6. How Leave Requests Work

Leave requests go through a simple approval process. Only pending leaves can be approved, rejected, or cancelled:

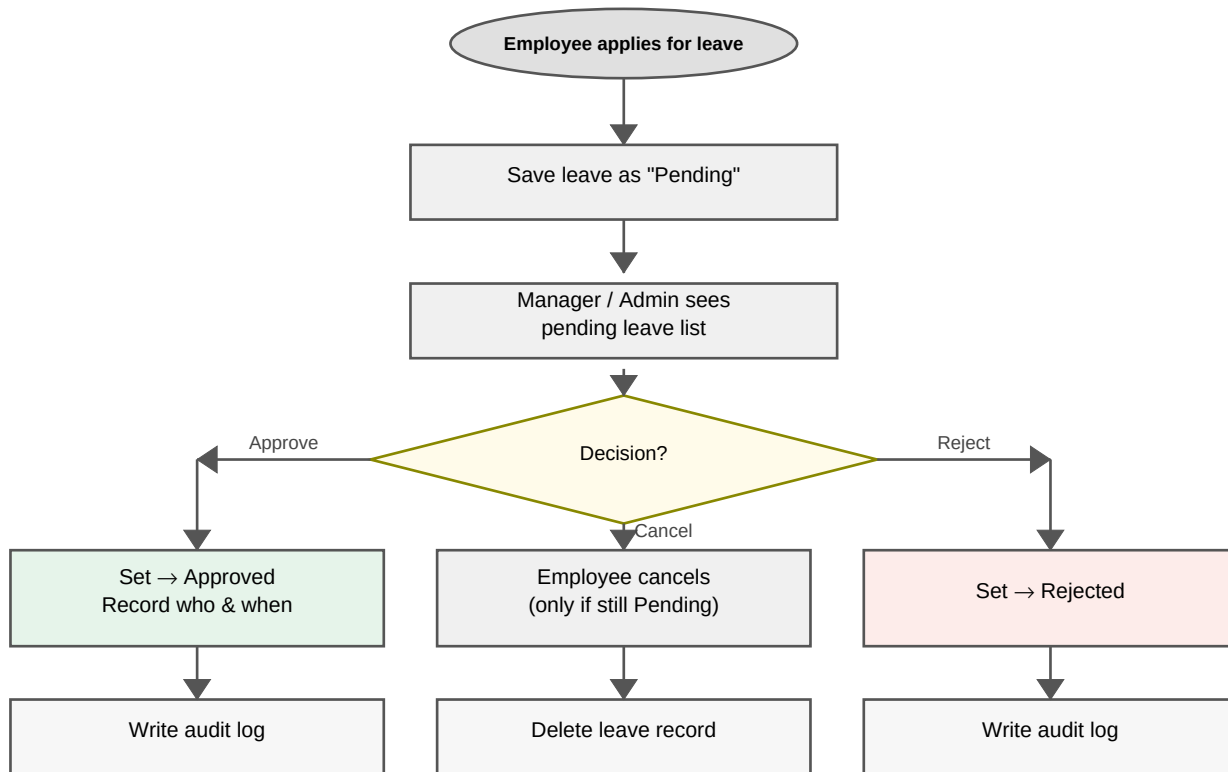


Figure 4 — Leave request lifecycle

7. What Each Dashboard Shows

Each role sees a different dashboard. The data is pulled from multiple tables and combined:

Role	What they see on their dashboard
Admin	Total employees, active projects, pending leaves, who is present today, recent leave requests, recently added employees

Manager	How many people are on their team, how many projects they manage, pending leaves from their team, who from the team
Employee	Their own check-in status for today, remaining leave balance, projects they are on, latest announcements

8. Security — How Access is Controlled

Every request (except login) must include a valid token. The backend checks three things on every request:

- Is the token valid and not expired?
- Is the token's signature correct?
- Does the user's role allow them to access this endpoint?

Security measure	How it works in simple terms
JWT Token	A signed pass with a time limit. Contains who you are and your role.
BCrypt Hashing	Passwords are scrambled in a way that cannot be reversed. Safe even if the database leaks.
Role-Based Access	Endpoints are restricted by role. Others get a 403 Forbidden error.
HTTPS	All data between browser and server is encrypted in transit.
Input Validation	All incoming data is checked before being processed.

9. Excel Report Generation

Admin can download reports as Excel files. The backend builds the file in memory and sends it as a download — nothing is saved on the server:

Report	What it contains
Employee	Full list of all employees with department and role
Attendance	Check-in / check-out records and total hours worked
Leave	All leave requests with status and approval info
Project	All projects with client, manager, and current status